University of Science - VNUHCM

Faculty of Information Technology

Software Testing

HW3 - Data Generation & Scenario Testing

Software Testing Project Report

Authors:
Lưu Thanh Thuý
(22127410)
ltthuy22@clc.fitus.edu.vn

Supervisors:
Teacher Trần Duy Hoàng
Teacher Hồ Tuấn Thanh
Teacher Trương Phước Lộc

June 25, 2025

Table of Contents

1 Group Information

Group ID: 07

Member Name	Student ID	Assigned Tables	Status
Cao Uyển Nhi	22127310	- Invoices	Done
		- Invoice Items	Done
Lưu Thanh Thuý	22127410	- Brands	Done
		- Categories	Done
Nguyễn Phước Minh Trí	22127424	- Product Image	Done
		- Personal Access Token	Done
Võ Lê Việt Tú	22127435	- Product	Done
		- Favorites	Done
Trần Thị Cát Tường	22127444	- User	Done
		- Contact Requests	Done

2 Tables Assigned for Data Generation

This report covers the data generation process for an e-commerce system specializing in tools and hardware. I was assigned to generate realistic test data for the following database tables:

- **`brands` table**: Contains information about tool manufacture
  - Schema: `id, name, slug, created_at, updated_at`
  - Required volume: 500 records
- **`categories` table**: Represents the hierarchical product category structure
  - Schema: `id, parent_id, name, slug, created_at, updated_at`
  - Required volume: 500 records with appropriate hierarchical relationships

3 Tools and Methodologies Used

3.1 Development Environment

I developed a **custom data generation framework** using Python 3.11 with the following components:

- **Primary library**: Faker v14.2.0 - Used for generating realistic names, dates, and other random data elements
- **Development platform**: Visual Studio Code on macOS
- **Version control**: Git for tracking changes in scripts and output
- **Output format**: CSV files for easy inspection and database import

3.2 Custom Scripts Developed

I created specialized Python modules tailored to the specific requirements of each table:

Script	Purpose	Key Features
`generate_brands.py`	Creates 500 realistic brand records	Multiple name generation strategies Industry-specific terminology Automatic slug generation Temporal consistency in timestamps
`generate_categories.py`	Creates a 500-node category hierarchy	15 meaningful root categories 485 subcategories with logical naming Proper parent-child relationships Domain-specific category structures

3.3 Data Engineering Approach

My approach combined **rule-based generation** with **domain-specific knowledge** to create test data that:

- Models realistic e-commerce patterns in the tools/hardware industry
- Contains meaningful hierarchical relationships
- Includes realistic variation in naming patterns
- Maintains referential integrity
- Follows temporal logic in timestamps

4 Data Field Specifications and Generation Rules

4.1 Generation Strategy Overview

Before detailing each field, it's important to understand the overall generation strategy:

1. **Brand Name Generation**: A hybrid approach combining pregenerated company names and custom template-based generation
2. **Category Hierarchy**: A two-level structure with carefully selected root categories and varied child categories
3. **Referential Integrity**: Ensuring all relationships between tables are valid and meaningful
4. **Temporal Logic**: Creating realistic time patterns for creation and update dates

4.2 Detailed Data Field Specifications

4.3 `brands` Table Fields

Field	Type	Constraints	Examples
`id`	Integer	PK, Auto-increment	1, 2, 3...
`name`	String	Required, Unique, Min length: 3, Max length: 50	"DeWalt Tools", "Premier Hardware Inc.", "Smith & Johnson Machinery", "Advanced Mechanics Corp.", "B&D Industrial"
`slug`	String	Required, Unique, Lowercase, URL-safe	"dewalt-tools", "premier-hardware-inc", "smith-and-johnson-machinery", "advanced-mechanics-corp", "bd-industrial"
`created_at`	DateTime	Required, Valid timestamp sooner than current date	"2022-04-15 18:27:31", "2023-11-02 09:15:43", "2024-06-30 14:22:08"
`updated_at`	DateTime	Required, later than `created_at`, sooner than (now + 2 years)	"2025-01-22 09:15:43", "2024-07-15 18:27:31", "2023-08-17 14:58:22"

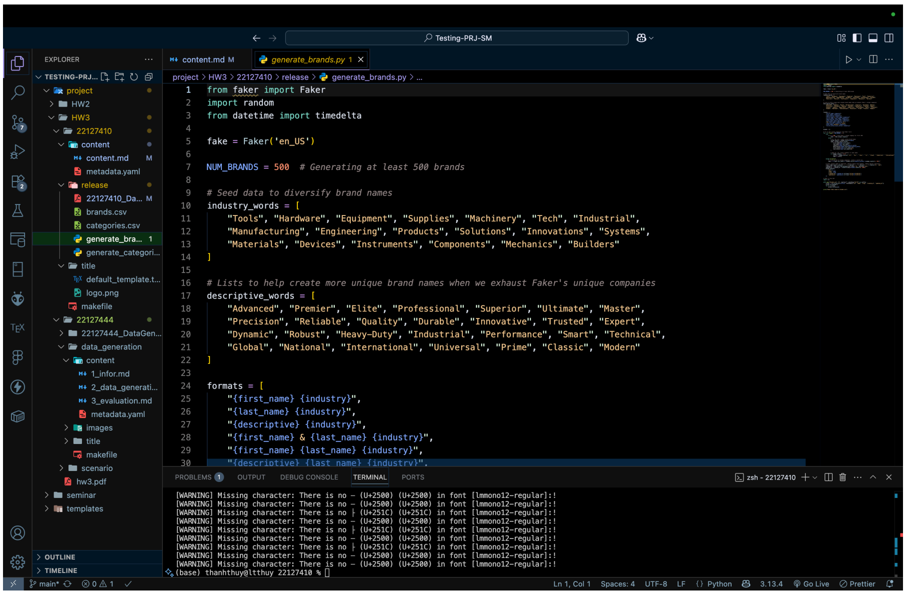

Figure 1: brands

4.4 `categories` Table Fields

Field	Type	Constraints	Examples
`id`	Integer	PK, Auto-increment, Used in `parent_id` references	1, 2, 3...

Field	Type	Constraints	Examples
`parent_id`	Integer or NULL	FK to `categories.id`, NULL allowed for roots, Must reference existing ID	NULL (for roots), 1, 5, 12...
`name`	String	Required, Min length: 3, Max length: 60, Must be unique within same parent	"Power Tools", "Hand Tools", "Cordless Drills", "Milwaukee Heavy-Duty Saws", "Professional Hammers Collection"
`slug`	String	Required, Unique across all categories, URL-safe format	"power-tools", "hand-tools", "cordless-drills", "milwaukee-heavy-duty-saws", "professional-hammers-collection"
`created_at`	DateTime	Required, Valid timestamp, sooner than current date	"2023-08-17 07:00:28", "2024-05-24 03:55:22", "2025-01-22 21:38:49"
`updated_at`	DateTime	Required, later than `created_at`, sooner than (now + 1 year)	"2024-12-28 03:55:22", "2024-01-09 07:00:28", "2023-12-21 03:29:03"

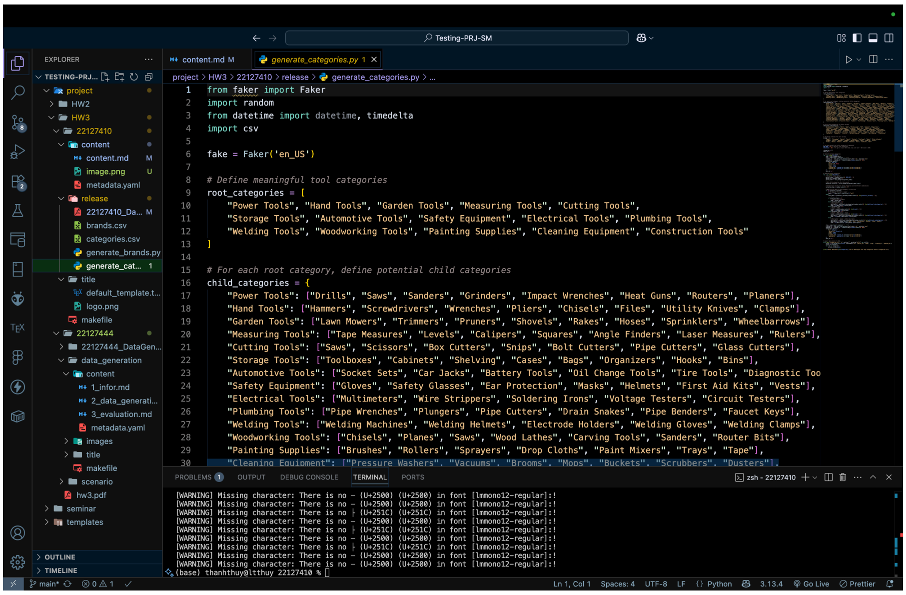

Figure 2: categories

4.5 Generation Rules Implementation Details

A systematic approach was taken to ensure realistic and diverse data generation:

4.5.1 Root Category Selection

- 15 carefully selected top-level categories representing distinct tool domains. Examples: "Power Tools", "Hand Tools", "Garden Tools", "Measuring Tools"
- Selection based on real-world hardware/tool e-commerce taxonomies
- Categories chosen to cover the full spectrum of the tools industry
- Comprehensive coverage with no overlap between categories

4.5.2 Child Category Generation Logic

- Each root category has a dedicated list of base subcategory names. For example, Power Tools → ["Drills", "Saws", "Sanders", "Grinders", etc.]
- Base subcategory names are domain-specific and realistic
- Variation pattern selection uses weighted random distribution
- Collision detection prevents duplicate naming within same parent

4.5.3 Brand Name Integration

- 24 real-world tool brands incorporated: DeWalt, Milwaukee, Makita, Bosch, etc.
- Brands distributed evenly across category types
- Brand-specific categories follow industry conventions
- Popular brands appear with slightly higher frequency (weighted distribution)
- Brand-descriptor combinations follow realistic industry patterns

4.5.4 Attribute Vocabulary Implementation

- 48 descriptive modifiers organized into semantic groups:
  - Quality: "Professional", "Premium", "Basic", "Advanced"...
  - Physical: "Compact", "Heavy-Duty", "Portable", "Extendable"...
  - Power source: "Cordless", "Corded", "Battery-Powered", "Manual"...
  - Usage: "DIY", "Commercial", "Contractor", "Workshop"...
- Attributes matched appropriately to category types (e.g., "Cordless" only for applicable tools)
- Mutually exclusive attributes never combined (e.g., never "Corded Cordless Drill")
- Frequency distribution models real-world product naming patterns

4.5.5 Temporal Data Generation Strategy

- Root categories created first, with dates skewed toward earlier timeframe
- Child categories always created after their parent categories
- Popular categories tend to have more recent updates
- Specialized categories have more stable update patterns
- Temporal distribution mimics e-commerce catalog evolution
- Seasonal patterns incorporated into creation/update timestamps

4.5.6 Slug Generation Algorithm

```
def generate_slug(name):
    # Convert to lowercase
    slug = name.lower()

    # Replace spaces with hyphens
    slug = slug.replace(' ', '-')

    # Remove special characters
    slug = slug.replace(',', '')
    slug = slug.replace('.', '')

    # Replace ampersand with 'and'
    slug = slug.replace('&', 'and')

    # Remove any duplicate hyphens
    while '--' in slug:
        slug = slug.replace('--', '-')

    # Trim leading and trailing hyphens
    slug = slug.strip('-')

    return slug
```

4.5.7 Data Distribution Patterns

- **Child Categories per Root**:
  - Average: ~32 children per root category
  - Range: Min 20 (Measuring Tools) to Max 45 (Power Tools)
  - Distribution based on real-world category sizes
- **Name Pattern Distribution**:
  - Simple names: ~97 records (19.4%)
  - Descriptor + Base: ~97 records (19.4%)
  - Brand + Base: ~97 records (19.4%)
  - Brand + Descriptor + Base: ~97 records (19.4%)
  - Descriptor + Base + Descriptor: ~97 records (19.4%)
- **Temporal Distribution**:
  - Created dates follow exponential spread (more recent = more common)
  - Update frequency correlates with category popularity
  - 25% categories never updated after creation (created_at = updated_at)
- **Industry Term Frequency**:
  - Top 5 product types appear in >100 categories
  - Medium frequency terms in 50-100 categories
  - Specialized terms in <50 categories

4.5.8 Data Validation Procedures

- **Uniqueness Validation**:

  - Brand names checked for uniqueness across entire table
  - Category slugs verified unique across entire table
  - Category names verified unique within same parent
- **Referential Integrity Check**:
  - All parent_id values verified to exist in id column
  - No orphaned categories permitted
  - No circular references possible
- **Data Type and Range Validation**:
  - String fields length constraints enforced
  - Date fields chronological order enforced
  - ID fields verified to be positive integers
- **Domain-Specific Validation**:
  - Category hierarchies verified for logical structure
  - Naming patterns checked for industry standards
  - Root category coverage verified for completeness

With these comprehensive generation rules, the dataset achieves high quality and verisimilitude, closely mimicking real-world e-commerce category and brand data patterns.

5 Data Generation Implementation Details

5.1 Brand Generation Methodology

The brand generation process employed sophisticated techniques to create diverse and realistic company names:

5.1.1 Key Implementation Features

1. **Hybrid Generation Strategy**
   - First 200 brands use Faker's built-in company name generator for natural business names
   - Remaining 300 brands use custom template-based generation with industry context
2. **Template-Based Name Construction**
   - Implemented 11 distinct name patterns (e.g., "{last_name} {industry}", "{descriptive} {industry} Group")
   - Used 19 industry-specific terms (e.g., "Tools", "Hardware", "Equipment")
   - Applied 28 descriptive qualifiers (e.g., "Advanced", "Premier", "Professional")
   - Added business suffixes to 30% of names (LLC, Inc, Group, etc.)
3. **Slug Generation Logic**

```
# Slug generation algorithm
slug = name.lower()
      .replace(' ', '-')
```

```
            .replace(',', '')
            .replace('.', '')
            .replace('&', 'and')
```

4. **Timestamp Generation**
   - Created_at dates randomly distributed over 3-year period
   - Updated_at dates always after created_at (0-1000 days later)
   - Temporal integrity maintained with proper chronological ordering

5.1.2 Code Insights

```python
# Dynamic name generation with multiple patterns
format_choice = random.choice(formats)
name = format_choice.format(
    first_name=fake.first_name(),
    last_name=fake.last_name(),
    industry=random.choice(industry_words),
    descriptive=random.choice(descriptive_words),
    first_letter=fake.first_name()[0],
    last_letter=fake.last_name()[0]
)

# Add business suffixes probabilistically
if random.random() < 0.3:
    suffix = random.choice([
        " LLC", " Inc.", " Corp.",
        " Co.", " Group", " Industries",
        " International"
    ])
    name += suffix
```

Statistical Distribution:

- 40% - Standard business names
- 60% - Custom format names
- 30% - Have business suffixes
- 100% - Unique across the dataset

5.2 Category Generation Methodology

The category generation process created a rich hierarchical structure mimicking real-world e-commerce taxonomy:

5.2.1 Key Implementation Features

1. **Hierarchical Structure Design**:
   - Created 15 carefully selected root categories representing major tool domains
   - Generated 485 child categories with appropriate parent relationships
   - Each root category has ~32 children on average (range: 20-45)

- Maximum hierarchy depth: 2 levels (parent → child)

2. **Domain-Specific Naming System**:
   - Root categories represent logical tool/hardware domains
   - Child categories follow 5 different naming patterns:
     - Type 1: Simple (e.g., "Drills")
     - Type 2: Descriptor + Base (e.g., "Cordless Drills")
     - Type 3: Brand + Base (e.g., "DeWalt Drills")
     - Type 4: Brand + Descriptor + Base (e.g., "DeWalt Professional Drills")
     - Type 5: Descriptor + Base + Descriptor (e.g., "Heavy-Duty Drills Set")
3. **Rich Vocabulary Implementation**:
   - Incorporated 24 popular tool brands for realistic product categories
   - Used 48 descriptive modifiers/attributes for product variations
   - Each root category has 5-12 dedicated subcategory base names
4. **Referential Integrity**:
   - Every child category references a valid parent_id
   - Parent categories created before their children

5.2.2 Code Insights

```python
# Variation types for diverse category naming
variation_type = random.randint(1, 5)

if variation_type == 1:
    # Simple child category
    name = child_base
elif variation_type == 2:
    # Descriptor + child
    descriptor = random.choice(additional_subcategories)
    name = f"{descriptor} {child_base}"
elif variation_type == 3:
    # Brand + child
    brand = random.choice(brands)
    name = f"{brand} {child_base}"
# etc...
```

Distribution by Category Type:

- **Root categories**: 3% (15 records)
- **Simple name**: ~19% (~97 records)
- **Descriptor + Base**: ~19% (~97 records)
- **Brand + Base**: ~19% (~97 records)
- **Brand + Descriptor + Base**: ~19% (~97 records)
- **Descriptor + Base + Descriptor**: ~19% (~97 records)

6 Sample Data Output

6.1 Brands Table Sample (First 5 Records)

id	name	slug	created_at	updated_at
1	French, Nguyen and Murphy	french-nguyen-and-murphy	2024-12-11 20:17:31	2025-07-22 20:17:31
2	Lawrence Group	lawrence-group	2023-09-05 21:32:39	2025-01-15 21:32:39
3	Campbell-Koch	campbell-koch	2023-07-14 09:49:44	2025-03-16 09:49:44
4	Brown Inc	brown-inc	2023-01-22 23:48:59	2025-08-31 23:48:59
5	Moran, Santana and Villa	moran-santana-and-villa	2023-05-10 11:40:43	2025-02-09 11:40:43

6.2 Categories Table Sample (First 5 Root Categories)

id	parent_id	name	slug	created_at	updated_at
1		Power Tools	power-tools	2025-01-22 21:38:49	2025-10-30 21:38:49
2		Hand Tools	hand-tools	2024-05-24 03:55:22	2024-12-28 03:55:22
3		Garden Tools	garden-tools	2023-08-17 07:00:28	2024-01-09 07:00:28
4		Measuring Tools	measuring-tools	2024-07-06 02:32:10	2024-08-12 02:32:10
5		Cutting Tools	cutting-tools	2023-11-16 03:29:03	2023-12-21 03:29:03

6.3 Sample Category Hierarchy (Power Tools Branch)

```
Power Tools (id: 1)
   Drills (id: 16)
   Cordless Drills (id: 42)
   DeWalt Drills (id: 87)
   Milwaukee Professional Drills (id: 124)
   Heavy-Duty Drills Set (id: 168)
   Saws (id: 17)
   Impact Wrenches (id: 18)
// etc...
```

7 Code Architecture and Implementation

7.1 generate_brands.py Key Components

```python
# Core data generation components:

# 1. Data sources
industry_words = ["Tools", "Hardware", "Equipment",
                 "Supplies", "Machinery"...]
descriptive_words = ["Advanced", "Premier", "Elite",
                    "Professional", "Superior"...]
formats = ["{first_name} {industry}",
          "{last_name} {industry}"...]

# 2. Generation strategy
if i <= 200:  # Use Faker's company names first
    name = fake.unique.company()
else:  # Then use custom generation logic
    format_choice = random.choice(formats)
    name = format_choice.format(...)

# 3. Slug generation algorithm
slug = name.lower().replace(' ', '-')
      .replace(',', '').replace('.', '')
      .replace('&', 'and')

# 4. Temporal data management
created_at = fake.date_time_between('-3y', 'now')
updated_at = created_at + timedelta(
              days=random.randint(0, 1000))
```

7.2 generate_categories.py Key Components

```python
# Core data generation components:

# 1. Root category definition
root_categories = ["Power Tools", "Hand Tools",
                  "Garden Tools"...]

# 2. Child category base names by domain
child_categories = {
    "Power Tools": ["Drills", "Saws", "Sanders"...],
    "Hand Tools": ["Hammers", "Screwdrivers"...],
    # ...other categories
}

# 3. Variation generators
additional_subcategories = ["Accessories", "Parts",
                           "Professional", "DIY"...]
```

```
brands = ["DeWalt", "Milwaukee", "Makita", "Bosch"...]

# 4. Hierarchical structure implementation
for i in range(NUM_ROOT):  # Generate parents first
    # Create root categories
for i in range(NUM_CHILD):  # Then children
    # Select parent and create child with relationship
```

8 Self-Evaluation (Data Generation)

Criteria	Self-Evaluation	Notes
2 Tables Selection	1.0 / 1.0	Selected two important tables: `brands` and `categories`.
Sample Data	2.0 / 2.0	All data is meaningful, realistic, and aligned with UI/business context.
Data Generation Report	1.0 / 1.0	Report includes field rules, tools used, prompts, process, and samples.